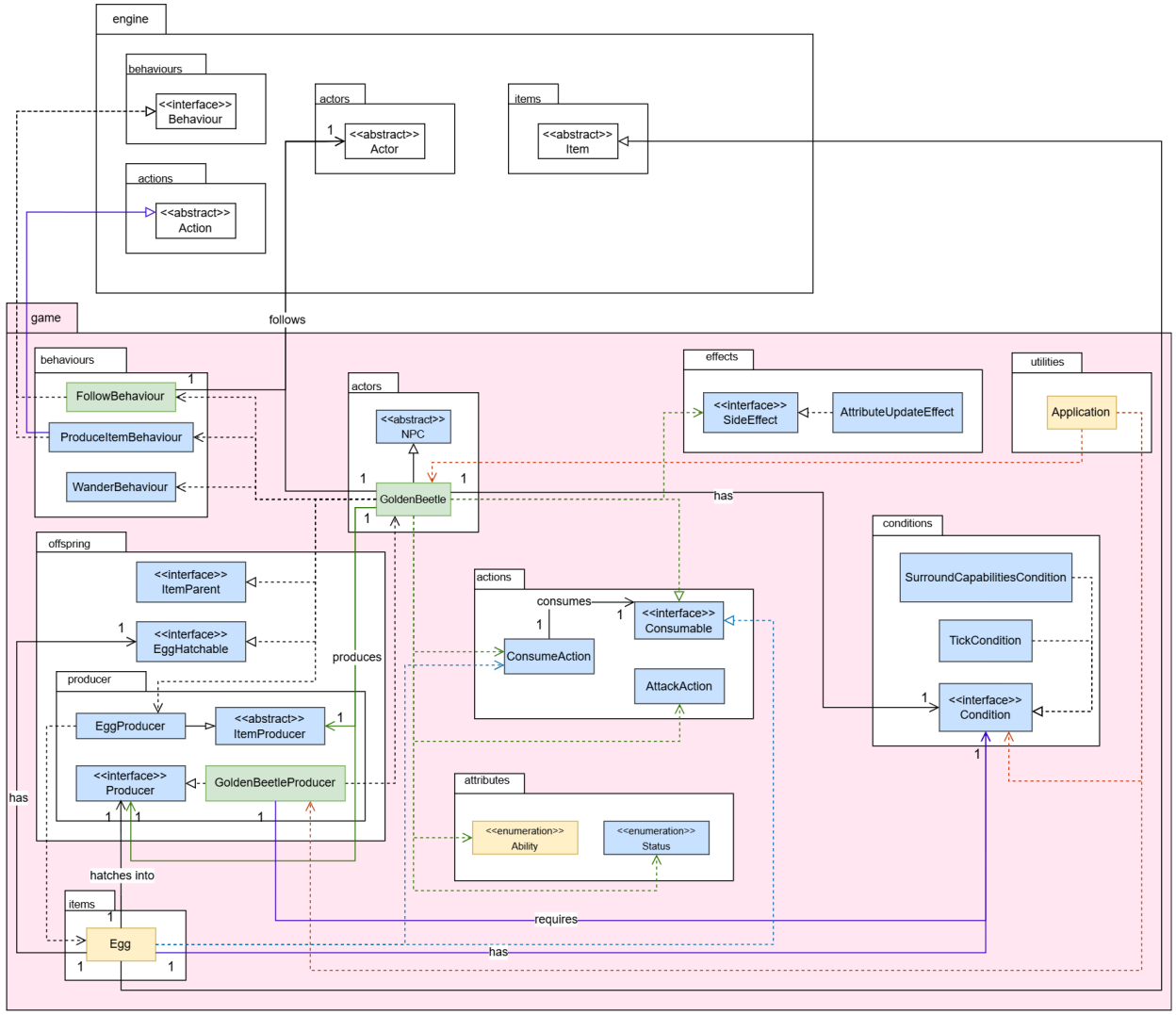
Design Documentation

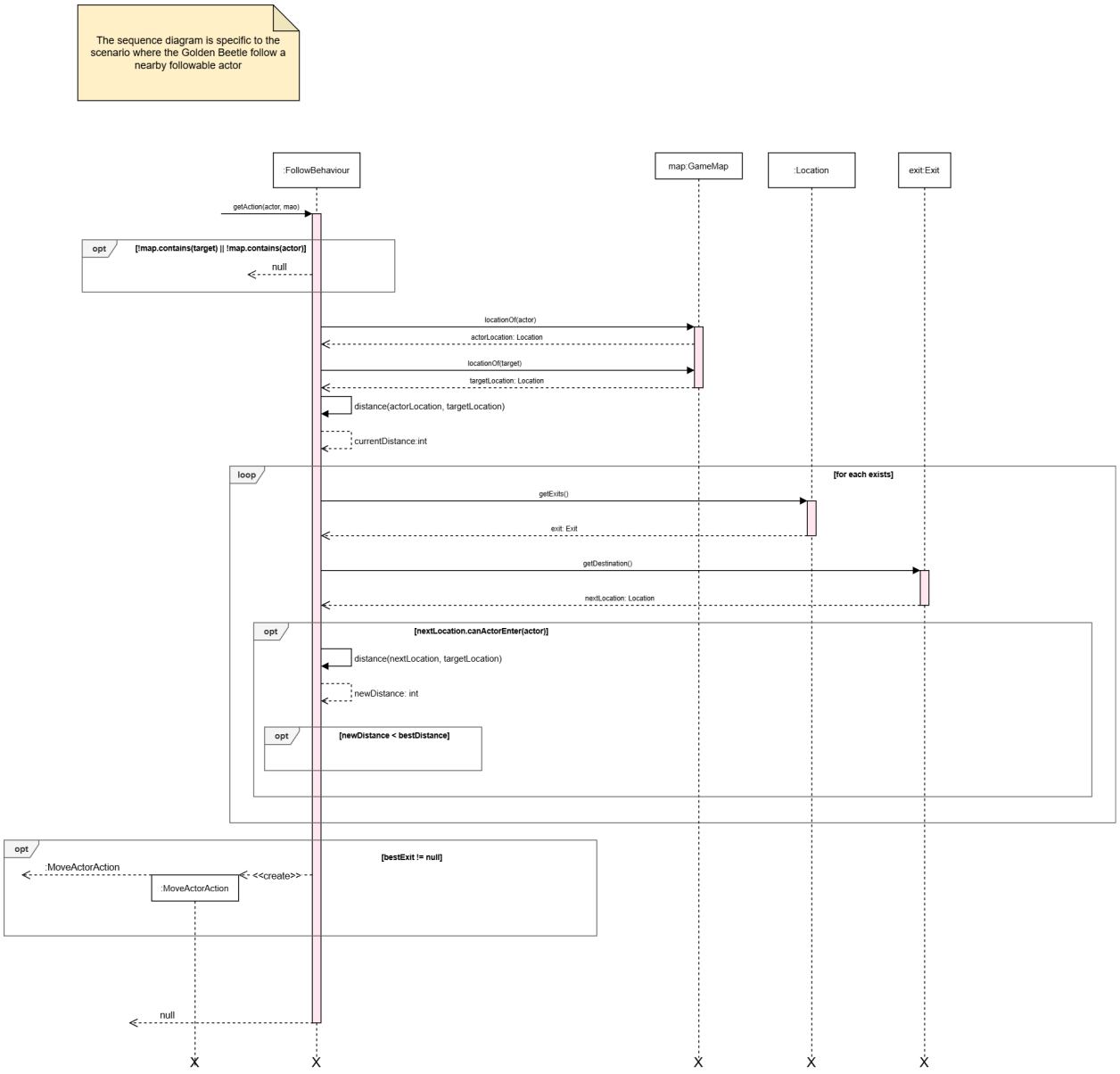
Req 2 - Be Wary of the Dung (Beetle)

UML diagram:



Existing Class	Blue
Modified Class	Yellow
New Class	Green

Sequence diagram:



Design Rationale:

Design 1:

Create CAN_BE_FOLLOWED ability in Ability enum class to represent that an actor can be followed by another actor, particularly used in the Farmer class to allow creatures like GoldenBeetle to follow them.

Pros	Cons
Avoid using instanceof to check whether an actor is followable, reducing coupling between classes	Behaviour relying on CAN_BE_FOLLOWED requires consistent use across actor classes, otherwise follow logic may silently fail
Makes behaviour eligibility clear by representing followable status as an ability	
New actor types can be marked as followable just by adding this ability without needing to change follow logic	

Design 2:

Create FollowBehaviour class which implements Behaviour interface from the engine. This class allows an actor to follow a specific target by moving one step closer each turn based on Manhattan distance. It is used in GoldenBeetle to follow followable actors such as Farmer.

Pros	Cons
Encapsulates following logic into a dedicated class, separating it from other actor behaviours	Only checks immediate adjacent tiles and one-step movement, which does not support long-range following or obstacle avoidance
Can be reused across different actors that follow targets without repeating movement code	
Can be added into an NPC’s behaviour map and executed in playTurn() without custom logic	

Design 3:

Create a GoldenBeetleProducer class to create a new GoldenBeetle instance based on reproduction conditions. This class implements the Producer interface and overrides both

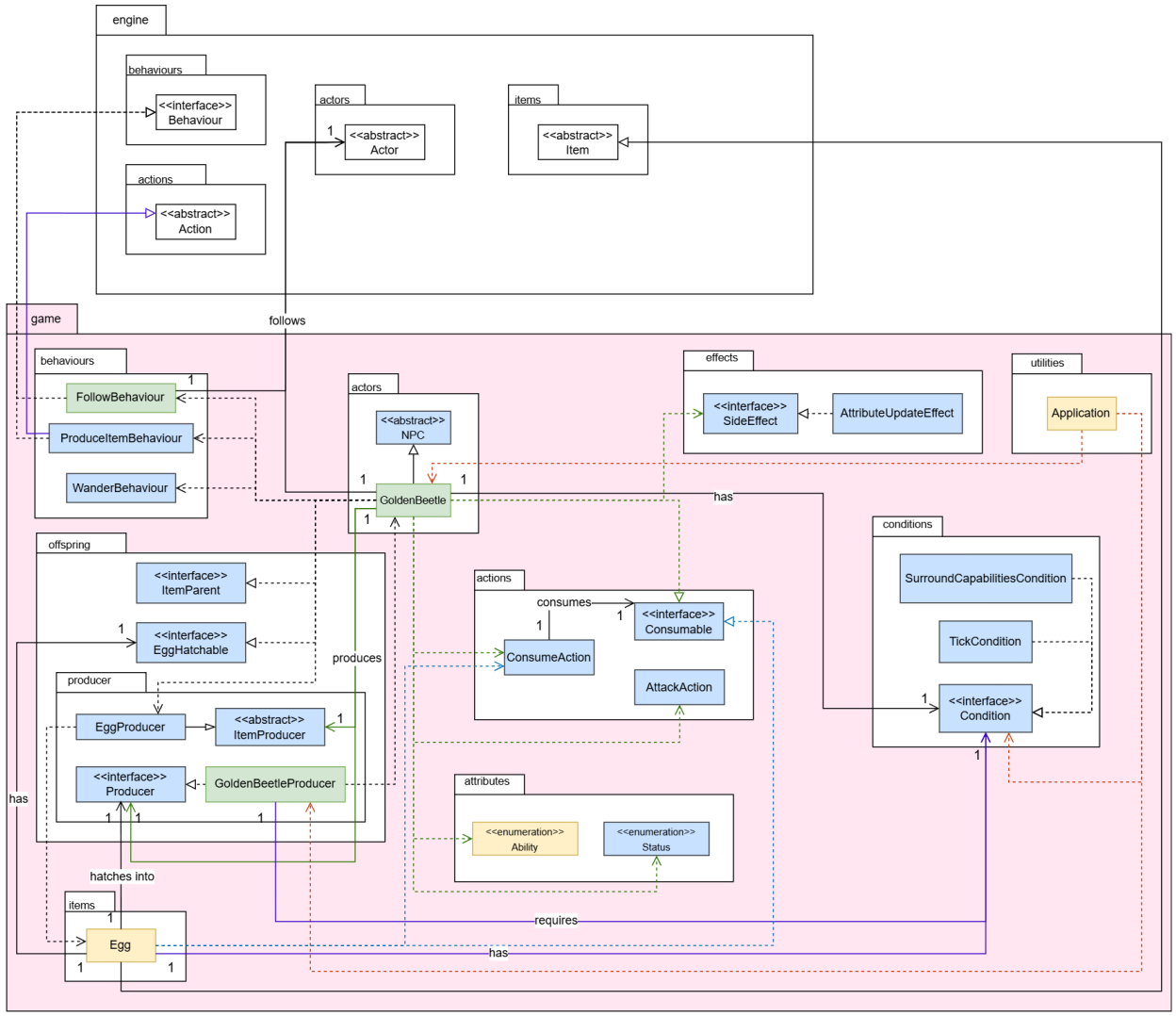
create() methods to support different ways of creating the beetle (eg. egg hatch or offspring logic)

Pros	Cons
Encapsulates creation logic for GoldenBeetle, keeping it separate from usage sites	May lead to too many producer classes if each creature needs its own version with only minor changes
Supports both direct reproduction and egg-based hatching through overloaded create() methods	
Can be used polymorphically in any generic producer context (eg. safely substitutable when Producer instance is expected)	

Design 4:

Create GoldenBeetle class to represent a new creature in the valley. This class extends NPC and implements ItemParent, EggHatchable and Consumable interfaces to support egg-laying, beetle consumption and golden egg consumption logic. It uses TickCondition for turn-based reproduction and allows direct consumption by the Farmer for health and rune gain.

Pros	Cons
Overloaded constructors allow the beetle to be used in both offspring and egg-laying scenarios without duplicating logic	Requires external setup to pass the correct reproduction and hatching conditions when creating the beetle
Supports egg production, movement and following by using separate behaviour classes	Behaviour priority must be carefully managed to ensure correct execution order, which may lead to maintenance overhead
Implements Consumable and overrides consume(), allowing the beetle to be used in ConsumeAction without depending on concrete class (avoid instanceof)	
Uses TickCondition to manage internal timing without exposing tick logic outside the class	



The diagram above shows the final design of Req2, which introduces a new creature called GoldenBeetle and reuses components from Req1 such as ItemParent, Consumable and EggHatchable. The GoldenBeetle class extends the engine's NPC class and adds behaviours using ProduceItemBehaviour, FollowBehaviour and WanderBehaviour. These behaviours are added through the beetle's behaviour map and executed inside playTurn(), allowing each behaviour to handle one type of logic. This follows the **Single Responsibility Principle (SRP)** since the beetle class does not manage reproduction or movement logic directly.

The GoldenBeetleProducer class implements the Producer interface and overrides two create() methods. One supports producing beetles from eggs, the other supports direct offspring creation. This avoids putting creation logic inside the beetle class and follows the **Open-Closed Principle (OCP)**. It also supports polymorphism, since any producer class can be passed around and used through the interface. This shows that the **Liskov Substitution Principle**

(LSP) is achieved. For example, an Egg can call create() without knowing the actual producer class, allowing egg logic to hatch into different creatures as long as their producers implement Producer. In this case, an OmenEgg can be configured to hatch into a GoldenBeetle. This reduces the connascence of algorithm as the logic is centralised and not duplicated.

The CAN_BE_FOLLOWED ability is added to the Ability enum to represent that an actor (e.g. Farmer) can be followed. This avoids checking specific types using instanceof and reduces tight coupling between GoldenBeetle and the actors it follows. Instead of hardcoding logic, the beetle checks if a nearby actor has the CAN_BE_FOLLOWED ability. This separates followability logic from follow behaviour logic, supporting **SRP**. It also allows new actors to become followable without changing beetle or behaviour classes, following **OCP**. However, this design depends on all followable actors correctly having the ability or the logic may silently fail. This introduces a connascence of meaning, as both GoldenBeetle and the actor must agree on what CAN_BE_FOLLOWED means.

FollowBehaviour is created to move the beetle one step closer to its target using Manhattan distance. This logic is separated into its own class, keeping it independent from the actor, which supports **SRP**. Since it implements Behaviour, it can be added into any actor's behaviour map and used like other behaviours. This shows **LSP** is achieved as all behaviours can be used polymorphically. The behaviour can be reused across multiple NPCs without rewriting movement logic, following the **Don't Repeat Yourself (DRY)** principle. Since following logic does not need to be changed for new actors, this also follows **OCP**.

The GoldenBeetle class implements Consumable and overrides consume() so the Farmer can consume it directly from the map. This allows it to be passed into ConsumeAction without knowing its specific class. Since ConsumeAction depends on the Consumable interface and not on concrete classes like GoldenBeetle, this follows the **Dependency Inversion Principle (DIP)**. The beetle uses TickCondition to track reproduction timing. This condition increases a counter every turn and checks when to produce an egg or offspring. By moving this logic into a separate class, the beetle does not manage reproduction ticks itself. This shows **SRP** is followed and since TickCondition is reused across creatures from REQ1, the design follows **DRY**. The behaviours are added in a specific order where ProduceItemBehaviour comes before FollowBehaviour to ensure egg production is prioritised. This introduces a connascence of position, as the outcome depends on behaviour order. However, this is managed within the constructor and does not affect other parts of the system.